

Generative Models

John Snyder

STAT 9340: Spring 2023

GANs

Generative adversarial networks (GANs) are an unsupervised generative model used for generating new samples that are similar to training examples.

- image/video applications: image translation, super-resolution, style synthesis
- audio: text-to-speech synthesis and music synthesis

GANs were introduced in 2014 (Goodfellow et al., 2014) as a minimax optimization problem or a zero-sum game in which two agents, a generator and a discriminator, compete with each other.

- 1 The generator is trained to produce fake examples that fool the discriminator.
- 2 The generator learns to synthesize samples which are indistinguishable from real data.

The generator's (fake!) examples serve as negative examples for the discriminator.

GANs



GANs

Basic idea: Set up a game between two “players,” one is the generator and one is the discriminator.

The discriminator considers samples and tries to determine if they are real or fake. It uses traditional supervised learning techniques to choose between two classes (real and fake).

The generator is trained to generate samples in order to fool the discriminator. The discriminator is a function D that takes input $\mathbf{x}$ and uses $\theta^{(D)}$ as parameters, where the generator is defined by a function G that takes $\mathbf{z}$ as input and uses parameters $\theta^{(G)}$. Both D and G are differentiable.

GAN: Game Theory, Nash Equilibrium

We have to assign cost functions for each player that are defined in terms of both sets of parameters. The discriminator seeks to minimize $J^{(D)}(\theta^{(D)}, \theta^{(G)})$ but must do so by only controlling $\theta^{(D)}$. Similarly, the generator seeks to minimize $J^{(G)}(\theta^{(D)}, \theta^{(G)})$ but must do so by only controlling $\theta^{(G)}$. Because each player's cost depends on the other player's parameters, but each player cannot control the other's parameters, this is said to be a game (as in game theory) rather than an optimization. That is, the solution to an optimization problem is a minimum, but the solution to a game is a Nash equilibrium.

Specifically, here we say that a Nash equilibrium is the tuple $(\theta^{(D)}, \theta^{(G)})$ that is a local minimum of $J^{(D)}$ with respect to $\theta^{(D)}$ and a local minimum of $J^{(G)}$ with respect to $\theta^{(G)}$.

GANs

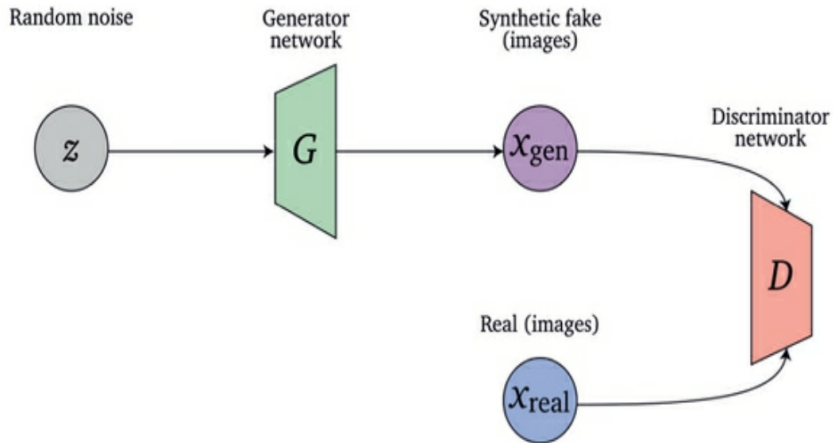
Generative adversarial networks (Goodfellow et al., 2014), were originally formulated as a minimax optimization problem.

$$\min_{\mathcal{G}} \max_{\mathcal{D}} V(\mathcal{G}, \mathcal{D}) = \mathbb{E}_x[\log \mathcal{D}(x)] + \mathbb{E}_z[\log(1 - \mathcal{D}(\mathcal{G}(z)))]$$

- $\mathcal{D}(x)$ is the discriminator's est. probability that real data x is real
- $\mathbb{E}_x$ is the expectation over the real data.
- $\mathcal{G}(z)$ is the output of the generator given random noise z .
- $\mathcal{D}(\mathcal{G}(z))$ is the discriminator's estimated probability that a synthesized sample is real
- $\mathbb{E}_z$ is the expectation over random noise input, that is over the generator's synthesized samples.

The goal of the generator is to generate a signal from random noise $z \sim P(z)$ in a way that it will be difficult for the discriminator to distinguish between the generated and real data $x \sim P_{\text{data}}$. The goal of the discriminator is to classify correctly real and generated data.

Image of GAN model



GAN: Cost Functions

Games designed for GANs have typically used the same cost function for the discriminator, $J^{(D)}$. They differ in the cost used for the generator. The discriminator cost is:

$$J^{(D)}(\theta^{(D)}, \theta^{(G)}) = -\frac{1}{2}E_{\mathbf{x} \sim p_{\text{data}}}[\log D(\mathbf{x})] - \frac{1}{2}E_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))] ,$$

which is the standard cross-entropy cost that is minimized when training a binary classifier with a sigmoid function. However, this is trained on two minibatches of data, one from the true dataset (which has label 1) and one coming from the generator (with label 0). Note, if we assume that p_{data} and p_{model} are nonzero everywhere, and minimize $J^{(D)}$ with respect to D , we can show (Goodfellow et al., 2014):

$$D(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{model}}(\mathbf{x})}$$

and estimating this ratio is an important part of a GAN (it depends on the ratio of these distributions).

Minimax

We also must specify a cost function for the generator. The simplest way to do this is by a zero-sum game, where the sum of all player's costs is always zero. So,

$$J^{(G)} = -J^{(D)}$$

In this case, since the two costs are directly tied together, we can summarize the entire game with a value function specifying the discriminator's payoff:

$$V\left(\theta^{(D)}, \theta^{(G)}\right) = -J^{(D)}\left(\theta^{(D)}, \theta^{(G)}\right)$$

Zero-sum games are called minimax games because their solution involves minimization in the outer loop and maximization in the inner loop:

$$\theta^{(G)*} = \arg \min_{\theta^{(G)}} \max_{\theta^{(D)}} V\left(\theta^{(D)}, \theta^{(G)}\right)$$

Notes on the zero sum game

Goodfellow (2017) notes that the zero-sum game is mostly of interest because it can be analyzed theoretically, but in practice it loses that advantage (the discriminator minimizes cross-entropy but the generator maximizes the same cross-entropy - this is unfortunate for the generator because when the discriminator successfully rejects the generator samples with high confidence, the generator's gradient tends to vanish.)

An alternative is to still use cross-entropy for the generator, but to flip the target used to construct the cost:

$$J^{(G)} = -\frac{1}{2} E_z [\log D(G(\mathbf{z}))]$$

Thus, in the minimax game the generator minimizes the log-probability of the discriminator being correct and in this game, the generator maximizes the log-probability of the discriminator being mistaken. (So, each player has a strong gradient when they are losing the game). [Note, this is not a zero-sum game and can't be described by a single value function.]

Training a GAN: Basic Algorithm (Goodfellow et al. 2014)

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log \left(1 - D(G(\mathbf{z}^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(\mathbf{z}^{(i)})) \right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Training Steps

Training a GAN is a bit different from training other models we have seen up to this point.

① Train the discriminator

- ① Generator creates fake data taking noise as an input
- ② Fake generator data is combined with real data
 - ① Data from $\mathbf{x} \sim p_{\text{data}}$ has a '1' label
 - ② Data from $G(\mathbf{z})$ has a '0' label
- ③ Train classification model on minibatches

② Train the generator

- ① Generate new data with generator.
- ② Assign '1' label to all, pass to discriminator.
- ③ Discriminator separates believable fakes from non believable
- ④ Backpropagate
 - ① Obviously, we would **not** update discriminator weights in this step.

③ Repeat the above two steps

Some practical tips and tricks for fitting GANs:

- train with labels (Semi-Supervised GAN)
- one-sided label smoothing - deep NNs often produce highly confident outputs that identify the correct class but with too extreme of a probability; in essence, one replaces highly confident probabilities with less confident ones (e.g., $1 - \alpha$ on the high end or $0 + \beta$ on the low end). This can be shown to act as a regularizer.
- feature matching - optimize the discriminator to inspect whether the generator's output matches the expected statistics of the real samples.
- minibatch discrimination - the discriminator digests the relationship between training data points in one batch, instead of processing each point independently. In one minibatch the closeness between every pair of samples, $c(\mathbf{x}_i, \mathbf{x}_j)$ is approximated and summed across all samples in the batch: $o(\mathbf{x}_i) = \sum_j c(\mathbf{x}_i, \mathbf{x}_j)$; then $o(\mathbf{x}_i)$ is added to the input of the model.

Practical tips and tricks (cont.)

- historical averaging - for both models, add $\left\| \theta - \frac{1}{t} \sum_{i=1}^t \theta_i \right\|^2$ into the loss function, where θ are the model parameters and θ_i corresponds to the parameter estimate at the past training iteration, i . This penalizes the training speed when θ is changing too fast.
- virtual batch normalization - each data sample is normalized based on a fixed batch (a “reference batch”) of data, rather than within its minibatch. The reference batch is chosen once at the beginning and stays the same through the training.
- adding noise - the distributions p_{data} and p_{model} are disjoint in high-dimensional space, which can lead to vanishing gradients. To artificially “spread out” the distribution so as to increase the chance that the two distributions overlap, one can add continuous noise to the inputs of the discriminator, D .

Some issues with GANs

- convergence: the model parameters oscillate, destabilize and never converge
- mode collapse: the generator collapses which produces limited varieties of samples,
- diminished gradient: the discriminator gets too successful so that the generator gradient vanishes and learns nothing,
- imbalance between the generator and discriminator: causes overfitting
- sensitivity to the hyperparameter selections

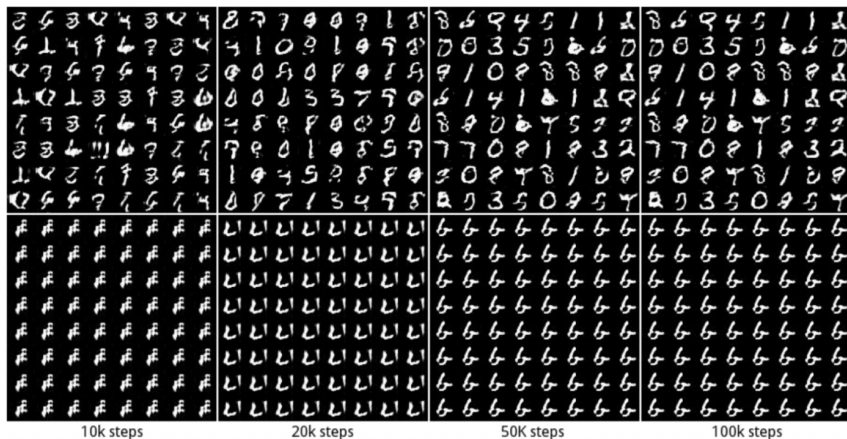
Convergence Issues with GANs

The biggest problem with GANs is convergence. Although one can usually set it up to get make progress (move “downhill”) for each player, one has to make progress towards the equilibrium solution - and so if one player successfully moves downhill on an update, it might actually move the other player uphill, and vice versa. This can occur indefinitely in some cases.

Mode Collapse: this occurs when the generator learns to map several different input values $\mathbf{z}$ to the same output point.

Typically, this happens in a “partial mode collapse” where the generator makes multiple images that contain the same color of texture themes, or multiple images containing different views of the same dog (for example).

Mode Collapse



A GAN in Tensorflow

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# MNIST Fashion
(x_train, _), (_, _) = keras.datasets.fashion_mnist.load_data()
#x_train = x_train.reshape(-1, 28 * 28) / 255.

x_train=x_train.reshape(-1,28,28,1)

x_train = x_train/255
x_train = x_train*2 - 1.

noise_size = 100 # Latent dimension used by generator
```

Simple TF GAN: Generator

```
noise_input = keras.Input(shape=(noise_size,))
x = keras.layers.Dense(512, activation=tf.nn.leaky_relu)(noise_input)
x = keras.layers.Dense(256, activation=tf.nn.leaky_relu)(x)
x = keras.layers.Dense(128, activation=tf.nn.leaky_relu)(x)
img = keras.layers.Dense(28 * 28)(x)
img = keras.layers.Reshape((28, 28, 1))(img)
generator = keras.Model(noise_input, img)
```

- Leaky ReLU is very useful in GANs so as to avoid vanishing gradients.

Simple TF GAN: Discriminator

```
img_input = keras.Input(shape=(28, 28))
x = keras.layers.Flatten()(img_input)
x = keras.layers.Dense(256, activation=tf.nn.leaky_relu)(x)
x = keras.layers.Dropout(0.75)(x)
x = keras.layers.Dense(128, activation=tf.nn.leaky_relu)(x)
x = keras.layers.Dropout(0.75)(x)
x = keras.layers.Dense(64, activation=tf.nn.leaky_relu)(x)
x = keras.layers.Dropout(0.75)(x)
validity = keras.layers.Dense(1, activation='sigmoid')(x)
discriminator = keras.Model(img_input, validity)
```

- In practice the discriminator needs to be heavily regularized

Simple TF GAN: GAN model

```
gan_input = keras.Input(shape=(noise_size,))  
img = generator(gan_input)  
gan_output = discriminator(img)  
gan = keras.Model(gan_input, gan_output)  
  
discriminator.compile(loss='binary_crossentropy', optimizer='a  
discriminator.trainable = False # Freeze the weights  
  
gan.compile(loss='binary_crossentropy', optimizer='adam')
```

Simple TF GAN: Training loop

We need to write training loop “manually.”

```
batch_size = 128
epochs = 60

totalSamples = (x_train.shape[0]//batch_size)
d_loss_real = np.zeros((epochs*totalSamples,))
d_loss_fake = np.zeros((epochs*totalSamples,))
g_loss = np.zeros((epochs*totalSamples,))
ratios = np.zeros((epochs,))

for epoch in range(epochs):
    for i in range(x_train.shape[0]//batch_size):
        lossArrInd = epoch*((x_train.shape[0]//batch_size)) + i
        # Get fake the real images
        noise = np.random.normal(0, 1, (batch_size, noise_size))
        fake_imgs = generator.predict_on_batch(noise)
        real_imgs = x_train[i*batch_size:(i+1)*batch_size]
        ....
```

Simple TF GAN: Training loop (cont)

Note: This code is all indented for the loop

```
...  
# Train the discriminator  
discriminator.trainable = True  
  
# Train on real images  
y = np.ones((batch_size,))  
d_loss_real[lossArrInd] = discriminator.train_on_batch(real_imgs,y)  
  
# Train on fake images  
y = np.zeros((batch_size,))  
d_loss_fake[lossArrInd] = discriminator.train_on_batch(fake_imgs,y)  
discriminator.trainable = False  
...
```

Simple TF GAN: Training loop (cont)

Note: This code is at the end of the epoch, outside the training loop

```
...  
    # Train the generator  
    noise = np.random.normal(0, 1, (batch_size, noise_size))  
    y = np.ones((batch_size,))  
  
    g_loss[lossArrInd] = gan.train_on_batch(noise, y)  
  
print('Checking ratio...')  
idx = tf.random.normal((len(x_train), noise_size))  
fake_images = generator.predict(idx, verbose=0)  
real_preds = discriminator.predict(x_train, verbose=0)  
fake_preds = discriminator.predict(fake_images, verbose=0)  
ratios[epoch] = (tf.reduce_mean(real_preds) / \  
    (tf.reduce_mean(real_preds)+tf.reduce_mean(fake_preds))).numpy()
```


Deep Convolutional GAN

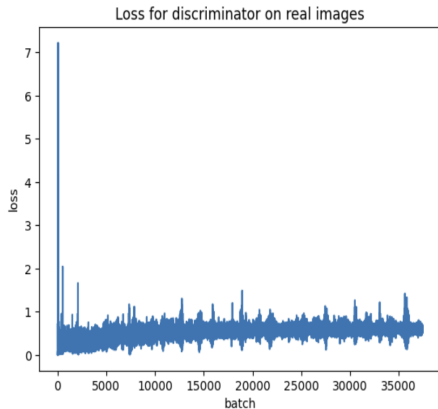
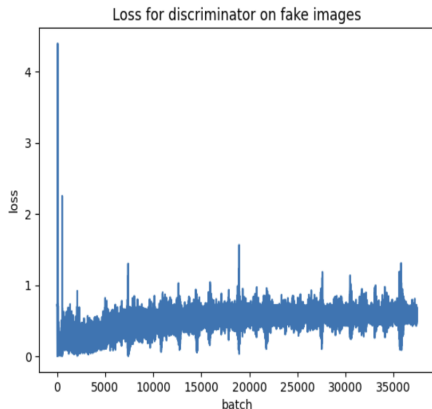
A DCGAN Radford et. al. 2016 is an extension of the GAN which uses convolutional and convolutional-transpose layers in the discriminator and generator.

- convolutional-transpose layers does *upsampling* and thus increases the resolution of the image
- The values of padding and stride here are the one that hypothetically was carried out on the output to generate the input.

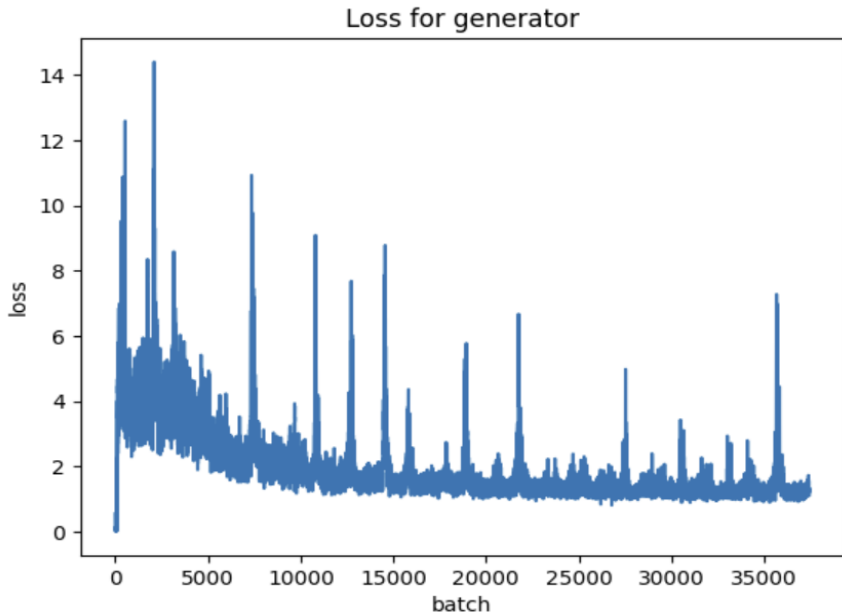
```
noise_input = keras.Input(shape=(noise_size,))
x = keras.layers.Dense(7 * 7 * 128)(noise_input)
x = keras.layers.Reshape((7, 7, 128))(x)
x = keras.layers.BatchNormalization()(x)
x = keras.layers.Conv2DTranspose(64, kernel_size=5, strides=2, padding="same",
                                activation="relu")(x)
x = keras.layers.BatchNormalization()(x)
img = keras.layers.Conv2DTranspose(1, kernel_size=5, strides=2, padding="same", activation="tanh")(x)
generator = keras.Model(noise_input, img)

img_input = keras.Input(shape=(28, 28, 1))
x = keras.layers.Conv2D(64, kernel_size=5, strides=2, padding="same", activation=tf.nn.leaky_relu)(img_input)
x = keras.layers.Dropout(0.5)(x)
x = keras.layers.Conv2D(128, kernel_size=5, strides=2, padding="same", activation=tf.nn.leaky_relu)(x)
x = keras.layers.Dropout(0.5)(x)
x = keras.layers.Flatten()(x)
validity = keras.layers.Dense(1, activation='sigmoid')(x)
discriminator = keras.Model(img_input, validity)
```

Simple TF GAN: Discriminator Loss



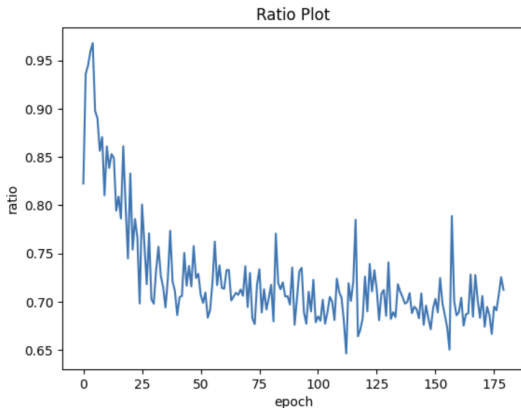
Simple TF GAN: Generator Loss



Simple TF GAN: Distribution Ratio

Recall:

$$D(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{\text{model}}(\mathbf{x})}$$

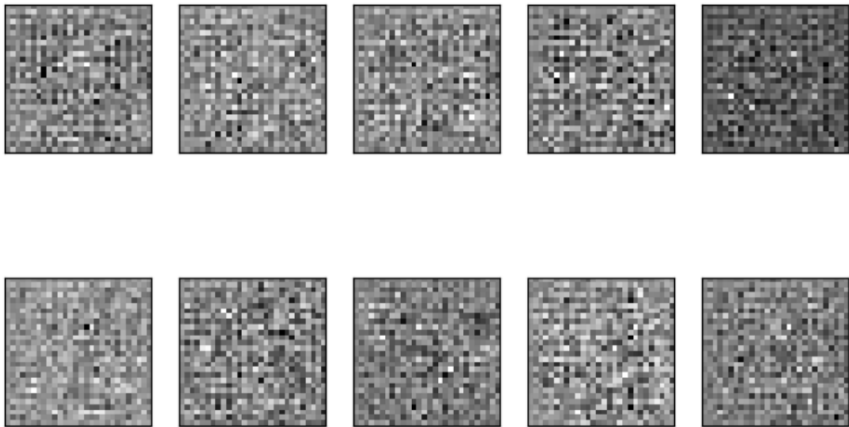


This indicates that our discriminator is likely too powerful.

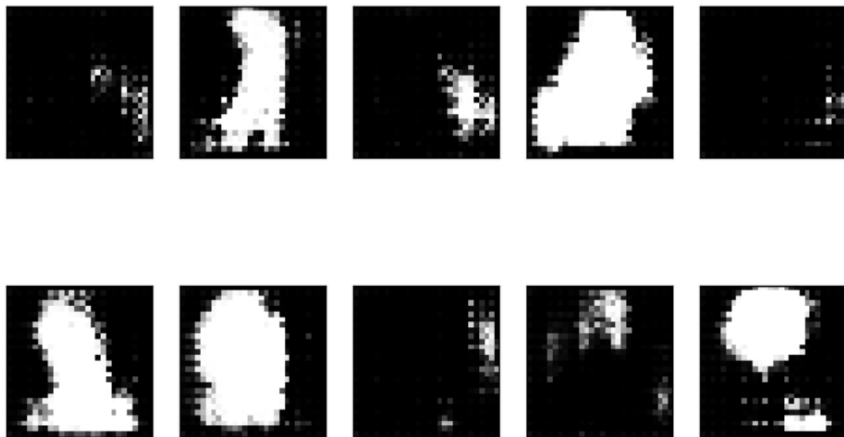
Actual images from this dataset



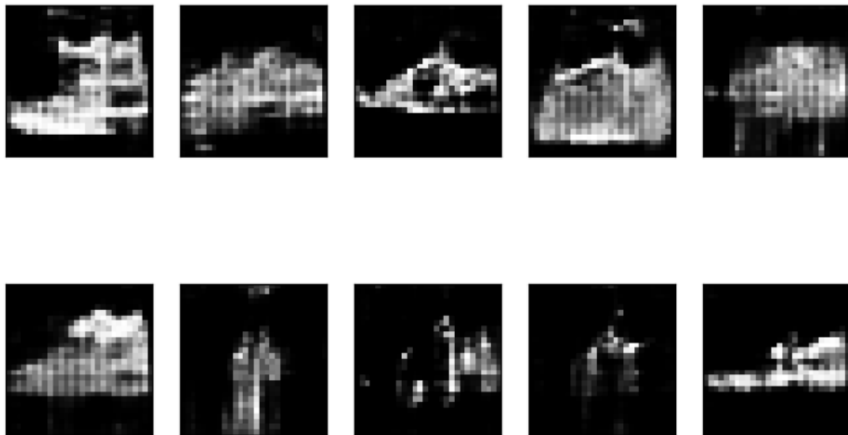
Generator performance prior to training



Generator performance after 1 epochs



Generator performance after 20 epochs



Generator performance after 180 epochs



Wasserstein GAN

GANs are known as hard to train because often, the generator is incapable of generating anything useful or produces the same thing (mode collapse). Wasserstein loss is one of the first principled approaches to deal with this and has become mainstream. The general idea is to replace the discriminator, which judges images as real or fake, by a critic, which just gives a score for each image. Unlike the judge, the critic gives an unbounded score. During training, the critic trains to score real images as much apart from fakes as it can while the generator trains to minimize this gap. The main advantage of this approach is that it is non-stationary in that the critic can change its views and score real images lower, or fake images higher. This makes the generator work harder to generate reals.

Note, the Wasserstein loss significantly improves convergence at the cost of requiring gradients to be 1-Lipschitz. To enforce this, the authors proposed to clip weights to $[-0.01, 0.01]$.

... and many more

- DCGAN: GAN with convolutions, useful for images
- Cycle GAN: learns mappings between two domains. Could be upsampling images, turning photos of horses to zebras, giving a photo the appearance of a famous artist, etc.
- Self attention GAN: Uses the attention mechanism to improve diversity of images

Variational Autoencoders (VAE)

Another way to generate images (or other dependent structures) from a sample of images is via variational autoencoders (VAEs).

Like with the generative model in a GAN, the main idea is to develop a low-dimensional latent space of representations where any point can be mapped to realistic looking structure (image). In GANs this is called a generator, but in VAEs it is the decoder.

As Chollet and Allaire (2018) summarize, VAEs are good for learning latent spaces that are quite structured, where specific directions encode a meaningful axis of variation in the data. GANs generate realistic images, but the latent space they come from may not have as much structure and continuity.

Variational Autoencoders (VAE)

Concept Vectors: Given a latent space of representations, certain directions in the space may encode interesting axes of variation in the original data. Chollet and Allaire (2018) present an example where for a latent space of images of faces, there may be smile vectors, such that if a latent point z is the embedded representation of a certain face, then the latent point $z + s$ is the embedded representation of the same face, smiling. Once you have identified such a vector, then you can edit images by projecting them into the latent space, moving their representation in a meaningful way, and then decoding them back to image space. Examples of concept vectors one may describe are adding sunglasses, removing glasses, etc. E.g., see Figure 8.11 in Deep Learning with R :



Variational Autoencoders (VAE)

VAEs were discovered in the mid 2010s as a AE that is particularly good at finding low-dimensional latent space representations of images and utilize variational Bayesian inference.

Recall, that traditional AEs would encode an image into a lower-dimensional space and then decode it back to the same image. Most commonly, constraints are imposed to ensure low-dimensionality and sparseness.

However, these AEs don't necessarily lead to well-structured latent spaces, nor are they particularly good at compression. By incorporating a variational learning component, VAEs can learn continuous, highly-structured latent spaces.

Variational Autoencoders (VAE)

Specifically, the VAE encoder actually compresses the image into parameters (mean and variance) of a distribution. Thus, it treats the image as a realization from a random process and tries to accommodate that uncertainty. The decoder then samples from this distribution. The stochasticity improves robustness and yields a latent space which encodes meaningful representations everywhere: every point sampled in the latent space is decoded to a valid output. Generally, here is how it works:

- 1 An encoder module turns the input samples into two parameters in a latent space of representations (means, and log-variances)
- 2 Randomly sample a point from the latent normal distribution that's assumed to generate the input image based on the means and log-variances
- 3 A decoder module maps this point in the latent space back to the original input image

Variational Autoencoders (VAE)

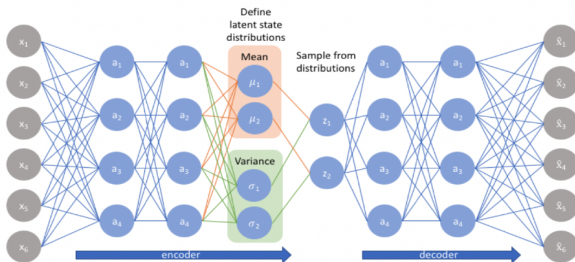
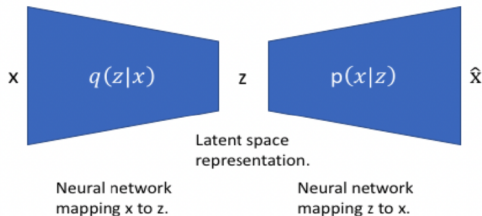
Continuity is achieved through the sampling from a continuous distribution. So, any two close points in the latent space will decode to highly similar images.

As Chollet and Allaire (2018) state, continuity, combined with the low dimensionality of the latent space, forces every direction in the latent space to encode a meaningful axis of variation of the data, making the latent space very structured and thus, highly suitable to manipulation via concept vectors.

The parameters of the VAE are trained with two loss functions: a reconstruction loss that forces the decoded samples to match the initial inputs, and a regularization loss that helps learn well-formed latent spaces and reduce overfitting to the training data.

Variational Autoencoders (VAE)

From Nabil Madali's blog (Feb 22, 2020: "From Autoencoder to Variational Autoencoder")



Variational Autoencoders (VAE)

A more statistical/mathematical presentation can be found in the excellent review by Kingma and Welling (2019). I briefly go over a bit of that here, but leave most of the details.

One key to the VAE is variational inference. Say we have an observation vector denoted by $\mathbf{x}$ and a latent random variable, $\mathbf{z}$. We are interested in the posterior $p_{\theta}(\mathbf{z} \mid \mathbf{x})$, given a particular likelihood and prior. As with standard variational Bayes methods, we find an approximating distribution (model) for the encoder inference model:

$$q_{\phi}(\mathbf{z} \mid \mathbf{x}),$$

where ϕ are the variational parameters. We seek $q_{\phi}(\mathbf{z} \mid \mathbf{x}) \approx p_{\theta}(\mathbf{z} \mid \mathbf{x})$. The encoder inference model can be parameterized in a number of ways, including using deep neural networks: e.g.,

$$\begin{aligned}(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) &= \text{EncoderNeuralNet } \phi(\mathbf{x}) \\ q_{\phi}(\mathbf{z} \mid \mathbf{x}) &= N(\mathbf{z}; \boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}))\dots\end{aligned}$$

Variational Autoencoders (VAE)

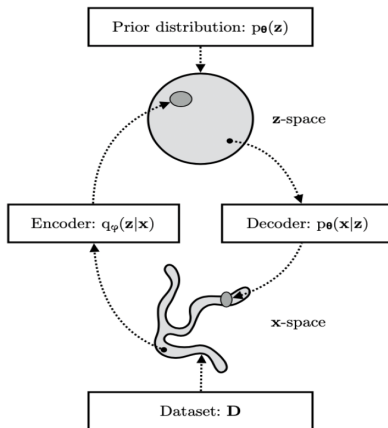


Figure 2.1: A VAE learns stochastic mappings between an observed $\mathbf{x}$ -space, whose empirical distribution $q_D(\mathbf{x})$ is typically complicated, and a latent $\mathbf{z}$ -space, whose distribution can be relatively simple (such as spherical, as in this figure). The generative model learns a joint distribution $p_{\theta}(\mathbf{x}, \mathbf{z})$ that is often (but not always) factorized as $p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})$, with a prior distribution over latent space $p_{\theta}(\mathbf{z})$, and a stochastic decoder $p_{\theta}(\mathbf{x}|\mathbf{z})$. The stochastic encoder $q_{\phi}(\mathbf{z}|\mathbf{x})$, also called *inference model*, approximates the true but intractable posterior $p_{\theta}(\mathbf{z}|\mathbf{x})$ of the generative model.

Variational Autoencoders (VAE)

To train this model, note that for any choice of $q_\phi(\mathbf{z} | \mathbf{x})$ and variational parameters, we can derive the ELBO and KL divergence:

$$\begin{aligned}\log p_\theta(\mathbf{x}) &= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x})] \\&= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{p_\theta(\mathbf{z} | \mathbf{x})} \right] \right] \\&= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z} | \mathbf{x})} \right] \right] \\&= \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \left[\frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right] \right]}_{=\mathcal{L}_{\theta, \phi}(\mathbf{x})} + \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \left[\frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z} | \mathbf{x})} \right] \right]}_{=D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x}))}\end{aligned}$$

The maximization of the ELBO w.r.t. θ and ϕ concurrently optimizes the two things we care about:

- 1 It will approximately maximize the marginal likelihood $p_\theta(\mathbf{x})$. This means our generative model will become better.
- 2 It will minimize the KL divergence of the approximation $q_\phi(\mathbf{z} | \mathbf{x})$ from the true posterior $p_\theta(\mathbf{z} | \mathbf{x})$, so $q_\phi(\mathbf{z} | \mathbf{x})$ becomes better.

Training VAEs

In practice, it helps to have two terms in the loss function used for training a VAE.

- 1 The *reconstruction loss*: Standard loss which measures how well the decoded output from the decoder network reconstructs the original input data.
- 2 *KL divergence loss*: This loss measures the difference between the distribution of the latent variables and a target prior distribution, which is usually a standard normal distribution. This acts as a regularizer, penalizing the latent parameters from straying too far from standard normals.

To start, note that the KL divergence between two multivariate gaussians P and Q can be written

$$\text{KL}(P||Q) = \frac{1}{2} \left(\text{tr}(\Sigma_Q^{-1} \Sigma_P) + (\mu_Q - \mu_P)^T \Sigma_Q^{-1} (\mu_Q - \mu_P) - k + \ln \frac{\det(\Sigma_Q)}{\det(\Sigma_P)} \right).$$

Training VAEs (cont)

If P is the learned distribution of the latent variables, with mean $\boldsymbol{\mu}_P = \mathbf{z}_{mean}$ and covariance matrix $\boldsymbol{\Sigma}_P = \text{diag}(\exp(\mathbf{z}_{logvar}))$, and Q is the standard normal prior distribution, with mean $\boldsymbol{\mu}_Q = \mathbf{0}$ and covariance matrix $\boldsymbol{\Sigma}_Q = \mathbf{I}$, the KL loss can be simplified as:

$$\text{KL}(P||Q) = \frac{1}{2} \left(\text{tr}(\boldsymbol{\Sigma}_P) + \boldsymbol{\mu}_P^T \boldsymbol{\mu}_P - k - \ln \det(\boldsymbol{\Sigma}_P) \right)$$

Which will give us the following expression to use the KL portion of the loss.

$$\text{KL}(P||Q) = -\frac{1}{2} \sum_{i=1}^k \left(1 + z_{log_var_i} - z_{mean_i}^2 - \exp(z_{log_var_i}) \right)$$

We can also multiply the constant in front of this KL divergence to control the strength of regularization.

Variational Autoencoders (VAE)

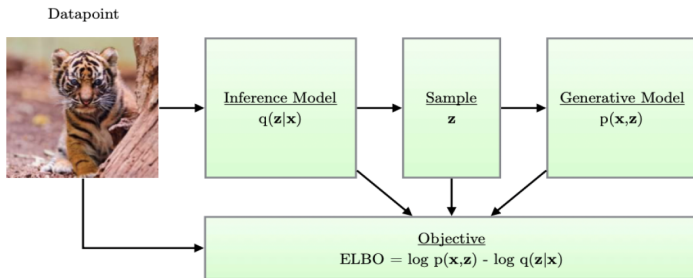


Figure 2.2: Simple schematic of computational flow in a variational autoencoder.

SGD can be used to optimize the ELBO jointly for ϕ and θ . See Kingma and Welling (2019) for details. In addition, they describe a “reparameterization trick” that allows the ELBO to be easily differentiated with respect to these parameters after a change of variables.

Variational Autoencoders (VAE)

After training, samples from $q_\phi(\mathbf{z} \mid \mathbf{x})$ are easily obtained because they are normals. One can then estimate the probability of data under the model using importance sampling Monte Carlo:

$$\log p_\theta(\mathbf{x}) \approx \log \frac{1}{L} \sum_{\ell=1}^L p_\theta(\mathbf{x}, \mathbf{z}^{(\ell)}) / q_\phi(\mathbf{z}^{(\ell)} \mid \mathbf{x})$$

Variational Autoencoders (VAE)

Challenges:

- Optimization Issues: stochastic optimization with the unmodified lower bound objective can get stuck in an undesirable stable equilibrium (one fix, the method of “free bits”)
- Blurriness of generative model: a consequence of the direction of the KL divergence; can be mitigated by choosing a sufficiently flexible inference model and/or sufficiently flexible generative model (see Kingma and Welling, 2019, for alternatives).

Obviously, like all other models we have talked about there are a tremendous number of variants of VAEs, with new ones coming out all of the time!